

Project Proposal Form, 2026
Institute of Information Technology (IIT)
University of Dhaka

Student's Name: RATUL SIKDER			
Student's Roll:	2506102	Phone:	01790532862

Project Title: Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

Motivation:

Modern software engineering teams dedicate an estimated 40–60% of their productive time to maintenance, debugging, and manual incident resolution rather than feature development. Automated pipelines — including CI/CD workflows, data integration jobs, and microservice deployments — fail silently and with alarming regularity. A schema change propagates upstream. A dependency version shifts. An API contract is violated. By the time engineers discover the failure, downstream damage is already extensive: corrupted data, broken analytics dashboards, stalled release cycles, and quantifiable financial loss.

Existing AI-assisted tooling such as GitHub Copilot, automated test generators, and deployment monitors address individual engineering tasks in isolation. None replicates the collaborative structure of a real engineering team, and none closes the loop between detecting a failure and autonomously repairing it. The engineer remains the central bottleneck — drawn into every incident, responsible for every fix, and required to approve every deployment manually.

Nexis addresses both deficiencies simultaneously. It models the software engineering team as a system of role-specialised AI agents and embeds a closed-loop self-healing intelligence that autonomously detects, diagnoses, synthesises repairs for, and validates failures. The vast majority of decisions are resolved automatically or escalated to the Administrator as a single one-click approval, reducing routine human involvement to the minimum necessary.

Project Description:

1. User Management & Intelligent Access Control

Nexis provides a complete authentication and role management layer. Users may register, log in, and manage their profiles with industry-standard security controls including bcrypt password hashing, JWT-based session tokens, and anomaly detection for unusual login events (atypical location, device, or access time). The system supports three distinct roles — Administrator, Engineer, and Viewer — each with scoped permissions and dedicated dashboard views. Role assignment is handled

intelligently: a behavioural analysis engine monitors access patterns and recommends role adjustments to the Administrator automatically. Role grants, revocations, and reassignments are managed through a centralised control panel with full audit visibility.

2. Admin Dashboard & Intelligent System Control Centre

The Administrator Dashboard serves as the authoritative command interface for the entire Nexis platform. Administrators can monitor all active agents, running pipelines, live incidents, and pending approvals in real time. The dashboard surfaces intelligent alerts — anomaly indicators, risk scores, and recommended remediation actions — generated automatically by the Self-Healing Loop. Routine low-risk decisions (minor patch approvals, non-critical rollbacks, health threshold adjustments) are resolved autonomously by the system and reported to the Administrator as completed actions. High-risk decisions are escalated with full contextual evidence and presented for one-click approval.

3. Dual-Layer Multi-Agent Architecture

Nexis operates across two fully implemented, co-ordinated agent layers connected by a shared memory bus and a Pydantic-typed inter-agent messaging protocol. Layer 1 — the Execution Team — consists of five role-specialised agents: the Architect Agent (system design and API contract governance), Backend Agent (code generation and repair), QA Agent (test generation and continuous validation), DevOps Agent (CI/CD orchestration and health monitoring), and Data Engineer Agent (data pipeline construction and schema management). Layer 2 — the Self-Healing Loop — consists of four monitoring agents: Sentinel (fault detection), Pathfinder (causal diagnosis), Synthesiser (repair generation), and Validator (shadow execution and approval queue management). All nine agents are fully built and deployed as part of this project, with every step instrumented, logged, and reported to the Administrator dashboard automatically.

4. Architect Agent — System Design & Contract Governance

The Architect Agent decomposes high-level requirements into a structured system blueprint, defining API schemas, data contracts, and service boundaries that constitute the system-wide source of truth. It actively monitors proposals from all other agents and automatically detects and blocks contract violations before they propagate downstream. Every violation is surfaced to the Administrator with a plain-language explanation and a recommended resolution, eliminating the need for manual investigation.

5. Backend Agent — Intelligent Code Generation & Autonomous Repair

The Backend Agent implements features against Architect-defined contracts using LLM-guided code synthesis, applying the minimal diff principle to generate the smallest valid change satisfying each requirement. When the Self-Healing Loop identifies a code-level failure, repair tasks are automatically delegated to the Backend Agent by the Synthesiser. Low-risk patches are applied and

deployed autonomously following shadow validation; high-risk changes are surfaced to the engineer for one-click approval with full supporting context.

6. QA Agent — Automated Property-Based Testing & Continuous Validation

The QA Agent auto-generates unit, integration, and property-based tests from API contracts and historical behavioural data, systematically covering edge cases that human-authored test suites routinely miss. Testing operates continuously rather than on demand, with structured defect reports fed back to the Backend Agent automatically. Failures are classified by severity: critical issues trigger immediate Self-Healing Loop activation; non-critical issues are batched and presented to the Administrator in a daily intelligent summary.

7. DevOps Agent — Deployment Orchestration & Autonomous Rollback

The DevOps Agent manages CI/CD pipeline execution via GitHub Actions and ArgoCD. It continuously monitors post-deployment signals — response latency, error rates, resource utilisation, and service uptime — and intelligently determines whether to trigger an automatic rollback or apply a targeted patch, based on failure severity and the available fix confidence scores. All decisions are logged and reported to the Administrator dashboard in real time; only high-stakes deployments require explicit human approval.

8. Data Engineer Agent — Schema Management & Drift Recovery

The Data Engineer Agent constructs and maintains ETL/ELT pipelines aligned with Architect-defined data contracts. It tracks schema versions continuously and propagates contract changes across all dependent pipelines. Schema drift — the most prevalent class of production failure — is detected automatically; a repair patch is synthesised and shadow-validated, then either applied autonomously for low-risk drift or queued for Administrator approval for breaking changes. All schema change events are logged with full lineage traceability.

9. Sentinel Agent — Proactive Fault Detection

The Sentinel Agent ingests telemetry continuously from the DevOps and Data Engineer Agents. It applies statistical process control over Apache Flink streaming infrastructure to identify the precise moment a failure occurs — within seconds rather than hours. Faults are classified by type and severity, a structured fault report is emitted, and the Pathfinder Agent is activated immediately. Administrators receive a real-time notification for every detected fault, including severity level and affected component, without the need to monitor logs manually.

10. Pathfinder Agent — Causal Fault Localisation

The Pathfinder Agent traverses the Neo4j system dependency graph using counterfactual causal inference (do-calculus, via DoWhy/EconML) to automatically distinguish root cause from

downstream symptoms. It produces a ranked hypothesis list with per-hypothesis confidence scores — for example: "Schema field user_id renamed in source system — 94% confidence." The diagnosis is delivered to the Administrator dashboard and passed to the Synthesiser Agent, replacing two to six hours of manual log investigation with a sub-minute automated result.

11. Synthesiser Agent — Autonomous Repair Generation

The Synthesiser Agent generates two to three candidate repair patches per fault via LLM-guided inductive program synthesis, formally constrained by system contracts and failure evidence. Each patch is automatically routed to the appropriate Layer 1 agent: schema fixes to the Data Engineer Agent, code patches to the Backend Agent. Every patch includes a plain-language explanation of the repair rationale. Simple, low-risk repairs are applied autonomously after shadow validation and reported as resolved to the Administrator; complex or high-risk repairs are escalated for explicit approval.

12. Validator Agent — Shadow Pipeline Execution & Patch Selection

The Validator Agent deploys each candidate patch into a Docker-isolated shadow pipeline running in real-time parallel with the production environment. It executes property-based test suites, compares shadow output against historical data invariants, and automatically selects the highest-confidence patch. Rejected patches are discarded with a diagnostic log entry. Approved patches are either deployed autonomously (low-risk) or presented to the engineer as a one-click approval complete with shadow test results, a diff view, and a plain-English summary — ensuring no unvalidated fix can reach production.

13. Intelligent Reporting & Notification System

Nexis generates automated reports summarising all agent activity, incidents detected, repairs applied, patches approved, and system health trends across any selected time window. Administrators receive a real-time notification feed and a daily intelligent summary with actionable insights. Engineers receive targeted alerts exclusively for incidents requiring their direct decision, eliminating alert fatigue caused by low-value notifications. All reports are generated automatically by the system with no manual compilation required and are available for export from the Administrator Dashboard.

Strategy & Timeline:

The development strategy for Nexis follows an iterative, modular approach structured across four sequential phases over 26 weeks. Phase 1 establishes the foundational infrastructure: system architecture, user authentication, role-based access control, and the Administrator Dashboard including its intelligence layer. Phase 2 builds the complete Self-Healing Loop — Sentinel, Pathfinder, Synthesiser, and Validator agents — together with the Backend and Data Engineer Agents and the shared memory and messaging infrastructure underpinning all agent communication. Phase 3 completes the Execution Team layer by implementing the Architect, QA, and DevOps Agents,

establishes the RLHF fine-tuning loop from engineer feedback, and performs end-to-end integration of all nine agents. Phase 4 delivers the intelligent reporting engine, the human approval interface, comprehensive fault injection and user study evaluation, and final submission. Every module is built with autonomous decision-making as the default; human judgement is requested only where risk or criticality demands it.

The project tasks breakdown is presented below:

Task No.	Task	Description	Dependencies
Phase 1 — Foundation, User System & Admin Interface (Week 1)			
1	System Architecture & Environment Setup	Define overall system architecture, agent communication protocol design, technology stack selection, repository structure, and CI/CD pipeline initialisation	None
2	User Registration & Core Authentication	Implement secure user registration, login, JWT-based token issuance, bcrypt password hashing, and email verification workflow	Task 1
3	Session Management & Account Recovery	Implement persistent session handling, token refresh strategy, secure password reset flow, and login anomaly detection (unusual device, location, or time flagging)	Task 2
4	Role-Based Access Control & Intelligent Role Recommendation	Implement Admin, Engineer, and Viewer role permissions; build behaviour-pattern analysis engine for automated role-change recommendations surfaced to Admin	Task 3
5	Admin Dashboard Core Interface	Build Admin control panel: real-time agent status, active pipeline feed, incident log, pending approval queue, and system health indicators	Task 4
6	Admin Dashboard Intelligence & Autonomous Reporting Layer	Integrate intelligent alert feed, risk-score overlays, autonomous decision reporting for low-risk actions, and high-risk escalation with one-click approval UI	Task 5
Phase 2 — Self-Healing Loop Core & Primary Execution Agents (Week 2 - 3)			
7	Shared Memory Bus & Inter-Agent Messaging Protocol	Implement Chroma vector store for episodic agent memory, PostgreSQL relational state layer, and Pydantic-typed	Task 1

		inter-agent contract schemas for all message exchanges	
8	Sentinel Agent: Streaming Fault Detection Engine	Build real-time telemetry ingestion pipeline using Apache Flink; implement statistical process control (SPC) models to detect anomalies within seconds of occurrence	Task 7
9	Sentinel Agent: Fault Classification, Severity Scoring & Admin Notification Integration	Implement multi-class fault taxonomy, severity weighting model, structured fault-report emission, and real-time Admin dashboard push notifications per detected event	Task 8
10	Pathfinder Agent: Dependency Graph Construction & Automated Lineage Tracking	Build Neo4j-backed system dependency graph; integrate OpenLineage for automated data and service lineage capture across all agent interactions	Task 9
11	Pathfinder Agent: Causal Inference Engine & Root Cause Ranking	Implement DoWhy/EconML do-calculus causal reasoning over the dependency graph; produce ranked hypothesis list with per-hypothesis confidence scores delivered to Admin	Task 10
12	Synthesiser Agent: LLM-Guided Patch Synthesis & Agent Delegation Logic	Implement constrained LLM program synthesis to generate 2–3 candidate repair patches per fault; build automated routing logic to delegate patches to the correct Layer 1 agent	Task 11
13	Validator Agent: Docker Shadow Pipeline & Property-Based Testing Integration	Build Docker-isolated shadow execution environment mirroring production in real time; integrate Hypothesis (Python) property-based test suites for patch validation	Task 12
14	Validator Agent: Autonomous Low-Risk Deployment & High-Risk Approval Queue	Implement confidence-threshold logic for autonomous patch deployment; build structured approval queue presenting diff, shadow results, and plain-English summary to Admin for high-risk fixes	Task 13
15	Backend Agent: LLM Code Generation & Minimal Diff Repair	Implement contract-driven LLM code synthesis applying the minimal diff principle; integrate Synthesiser delegation	Tasks 12, 14

		receiver for autonomous repair task handling	
16	Data Engineer Agent: ETL Pipeline Construction, Schema Drift Detection & Autonomous Repair	Build and validate ETL/ELT pipelines against Architect-defined data contracts; implement continuous schema version tracking, automated drift detection, repair synthesis, and Admin escalation for breaking changes	Tasks 12, 14
Phase 3 — Remaining Execution Agents, RLHF Loop & Full Integration (Week 4 - 6)			
17	Architect Agent: System Blueprint Generation, Contract Definition & Violation Enforcement	Implement requirement-to-blueprint decomposition; define API schemas and service boundaries as system-wide source of truth; build automated cross-agent contract violation detection and blocking with Admin reporting	Tasks 6–11
18	QA Agent: Property-Based Test Generation, Defect Reporting & Continuous Test Loop	Auto-generate unit, integration, and property-based tests from API contracts and historical behavioural data; implement continuous execution loop with severity-classified defect reports routed back to the Backend Agent	Task 17
19	DevOps Agent: CI/CD Orchestration, Runtime Health Monitoring & Intelligent Rollback Engine	Integrate GitHub Actions and ArgoCD for pipeline orchestration; implement continuous post-deployment signal monitoring (latency, error rates, resource utilisation); build decision engine for automated rollback versus targeted patch based on failure severity and fix confidence	Tasks 17, 18
20	RLHF Fine-Tuning Loop: Engineer Approval Feedback Pipeline & Model Adaptation	Build feedback capture pipeline from engineer approval, rejection, and modification history; implement reward model training and periodic LLM policy fine-tuning to improve patch quality over time	Tasks 9, 17–19
21	End-to-End System Integration: All 9 Agents, Admin Reporting Pipeline & Audit Trail	Perform full integration of all nine agents across both layers; validate end-to-end self-healing loop execution; verify Admin dashboard reporting, notification pipeline,	All tasks

		and complete audit trail for every autonomous action	
Phase 4 — Intelligence Layer, Evaluation & Final Submission (Week 7 - 8)			
22	Intelligent Reporting Engine & Real-Time Notification Feed	Build automated report generation for agent activity, incident summaries, repair outcomes, and system health trends; implement real-time Admin notification feed and scheduled daily intelligent summary with actionable insights	Task 21
23	Human Approval Interface: One-Click UI with Diff View, Shadow Results & Plain-English Explanation	Build engineer-facing approval interface presenting patch diff, shadow pipeline test results, causal diagnosis, and plain-English fix summary; enforce that no unvalidated change reaches production	Task 21
24	Fault Injection Testing: Schema Drift, API Contract Violations & Dependency Breakage Scenarios	Execute structured fault injection test suite covering all primary failure classes; measure self-healing loop activation latency, patch correctness rate, and false positive rate against manual-resolution baselines	All tasks
25	User Study, MTTR Measurement, Performance Optimisation & NASA-TLX Evaluation	Conduct NASA-TLX cognitive load study with engineer participants; measure Mean Time To Resolution (MTTR) reduction versus manual baseline; perform system-wide performance tuning and bottleneck resolution	Task 24
26	Final Integration Verification, Project Report, Presentation & Submission	Conduct full end-to-end regression test; finalise project report and evaluation results; prepare presentation slides; submit all deliverables	All tasks

Tech Stack to be used:

Frontend

- NextJS (Typescript)
- TanStack Query (server state)
- Zustand (client state)
- Tailwind CSS + shadcn/ui

Backend

- Python 3.12+
- FastAPI + Uvicorn (async REST + WebSocket)
- Celery + Redis (distributed task queue)
- Apache Kafka (event streaming backbone)
- Apache Flink (stateful stream processing + SPC)
- gRPC (inter-service agent communication)
- Redis (caching + pub/sub)

Memory & Database

- PostgreSQL + pgvector
- Chroma (vector store)
- Neo4j (dependency graph)
- Redis Stack (semantic caching)

AI & Agent

- LangGraph + AutoGen (multi-agent orchestration)
- Claude API (claude-sonnet-4) + GPT-4o
- LangSmith (agent tracing + evaluation)
- Pydantic v2 (inter-agent message contracts)
- DoWhy + EconML (causal inference)
- DSPy (prompt optimisation + compilation)
- Guardrails AI (output validation + structured synthesis)
- RAGAS (retrieval-augmented generation evaluation)

Observability & Validation

- OpenTelemetry (distributed tracing)
- Prometheus + Grafana (metrics + dashboards)
- Jaeger (trace visualisation)
- CUSUM / EWMA via Flink (statistical drift detection)
- Docker (hermetic shadow execution)
- Hypothesis + fast-check (property-based testing)
- Loki (log aggregation)

DevOps & Infrastructure

- Kubernetes + Helm
- ArgoCD (GitOps delivery)
- GitHub Actions (CI/CD)
- Vault (secrets management)

Supervisor's Name: Dr. Kazi Muheymin-Us-Sakib

Signature of the Supervisor: _____

Date: _____

Before the Midterm Presentation 1:

I confirm that the progress is satisfactory and I am forwarding it for the midterm presentation.

Signature of the Supervisor: _____

Date: _____

Proposal Presentation Feedback (if any):

Midterm 1 Presentation Feedback (if any):

Midterm 2 Presentation Feedback (if any):